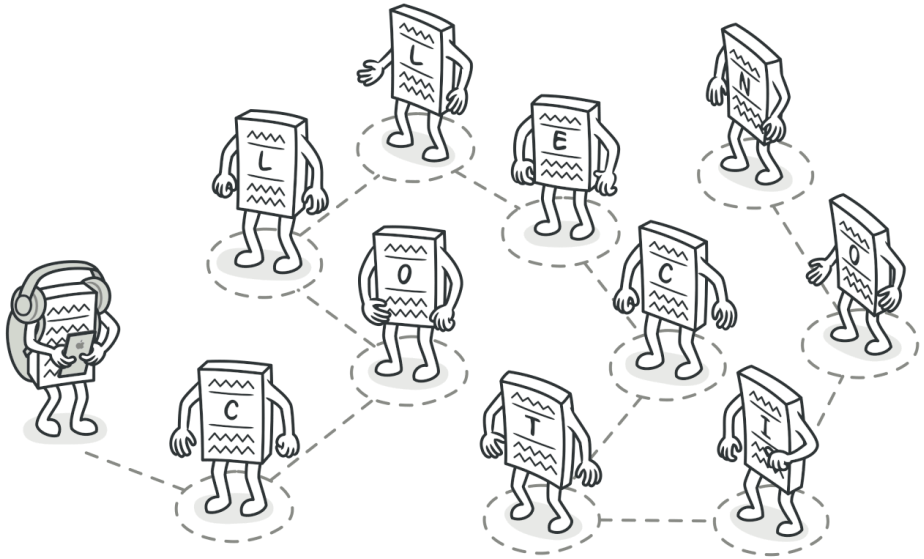
Поведенческие паттерны. Итератор



Паттерн Итератор
Iterator (Cursor)

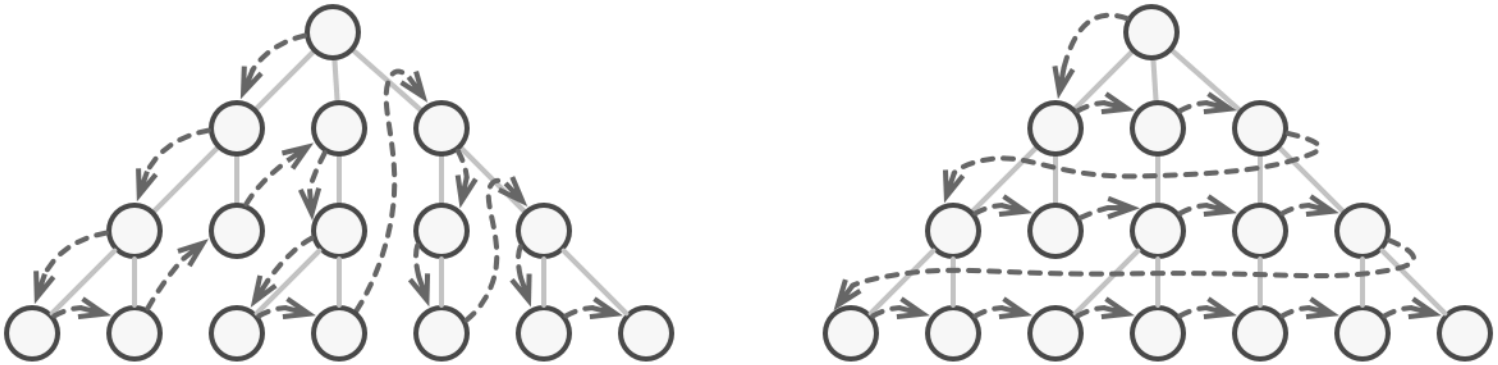


Итератор — это поведенческий паттерн проектирования, который даёт возможность последовательно обходить элементы составных объектов (*коллекций*), не раскрывая их внутреннего представления.

Благодаря **Итератору**, клиент может обходить разные коллекции одним и тем же способом, используя единый интерфейс итераторов.

Проблема

Задача коллекции – эффективное хранение данных

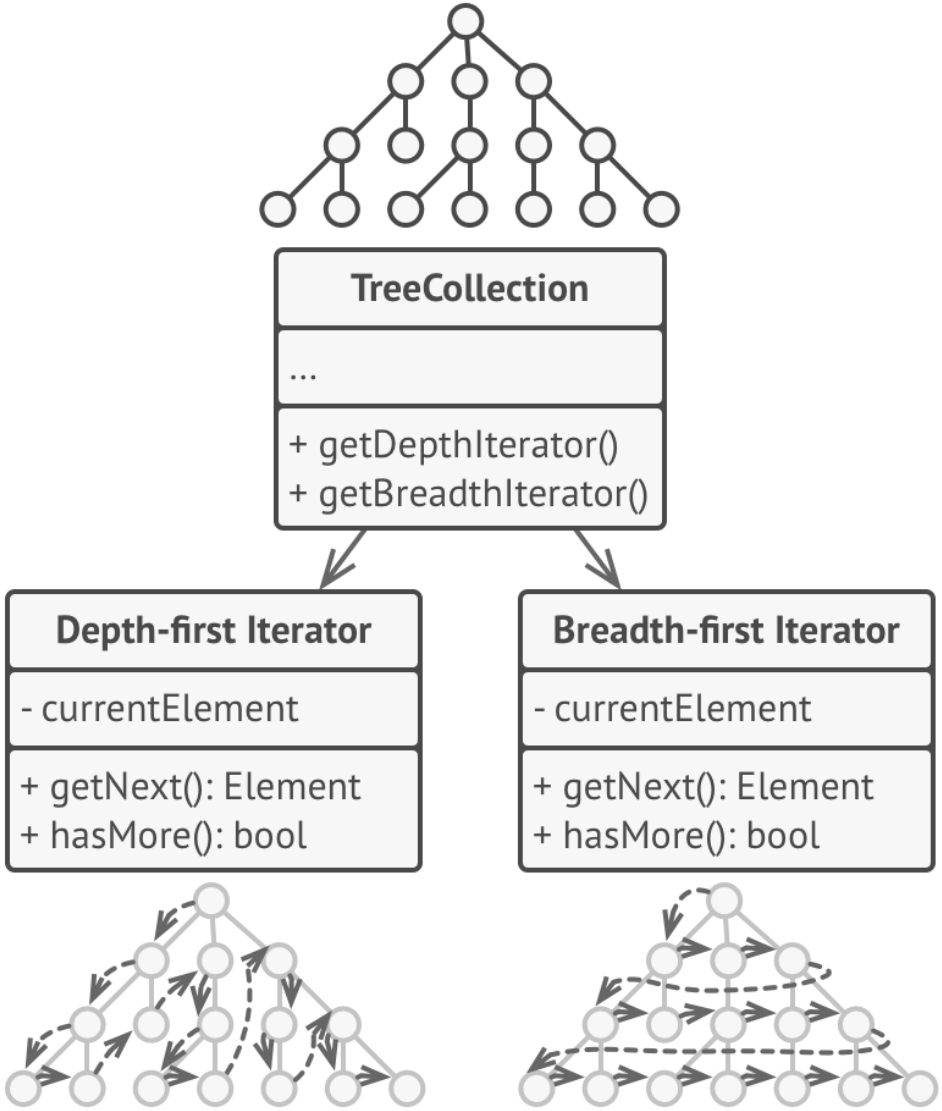


Одну и ту же коллекцию можно обходить разными способами.

Хранение алгоритмов обхода коллекции внутри кода коллекции – неэффективное решение, противоречит принципу *единственной ответственности*. Возникает тесная связь между коллекцией и алгоритмами доступа к ней.

Решение

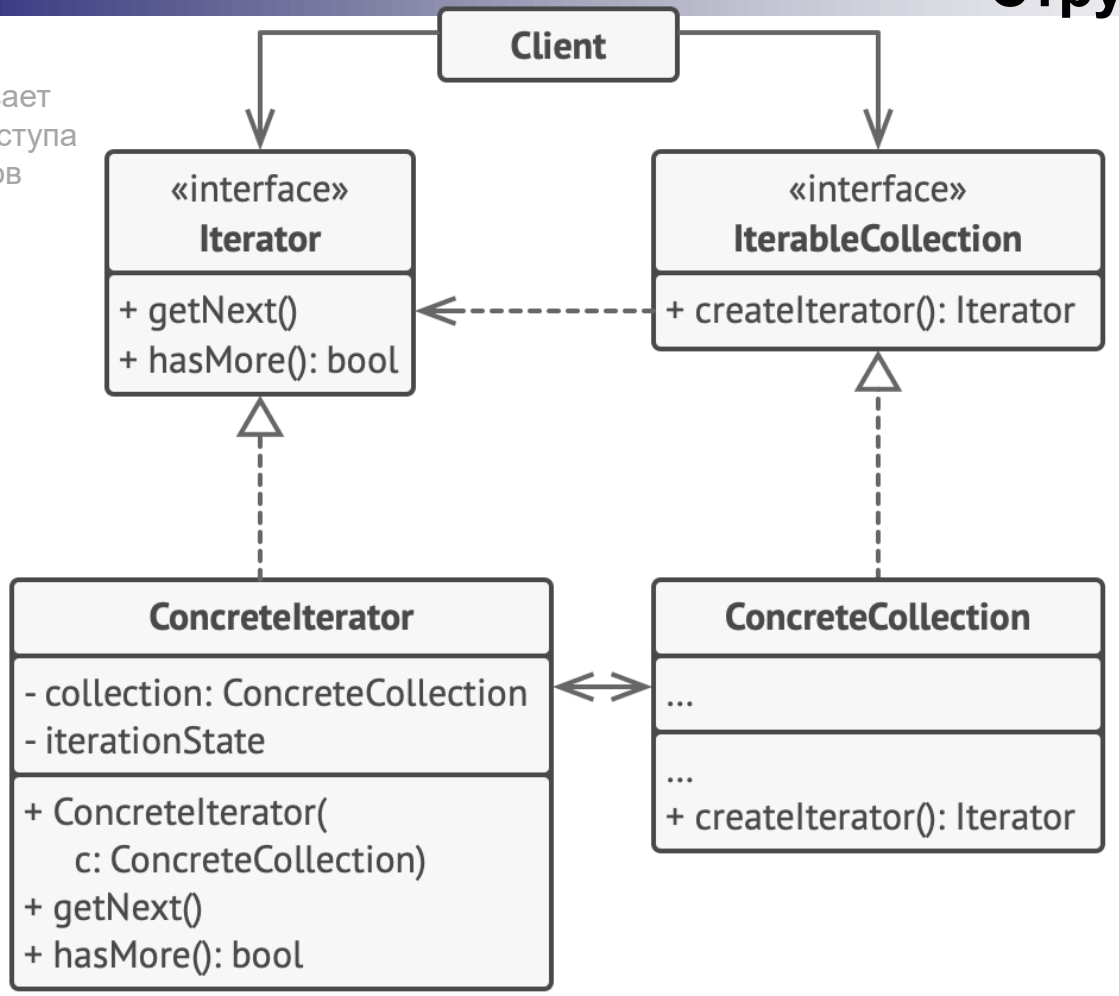
Паттерн **Итератор** предлагает вынести поведение обхода коллекции из самой коллекции в отдельный класс.



- ***Объект-итератор*** отслеживает состояние обхода, текущую позицию в коллекции и сколько элементов ещё осталось обойти.
- Одну и ту же коллекцию смогут одновременно обходить различные итераторы.
- Для добавления нового способа обхода, необходимо создать отдельный класс итератора, не изменяя существующий код коллекции.

Структура паттерна

[1]**Итератор** описывает интерфейс для доступа и обхода элементов коллекции.



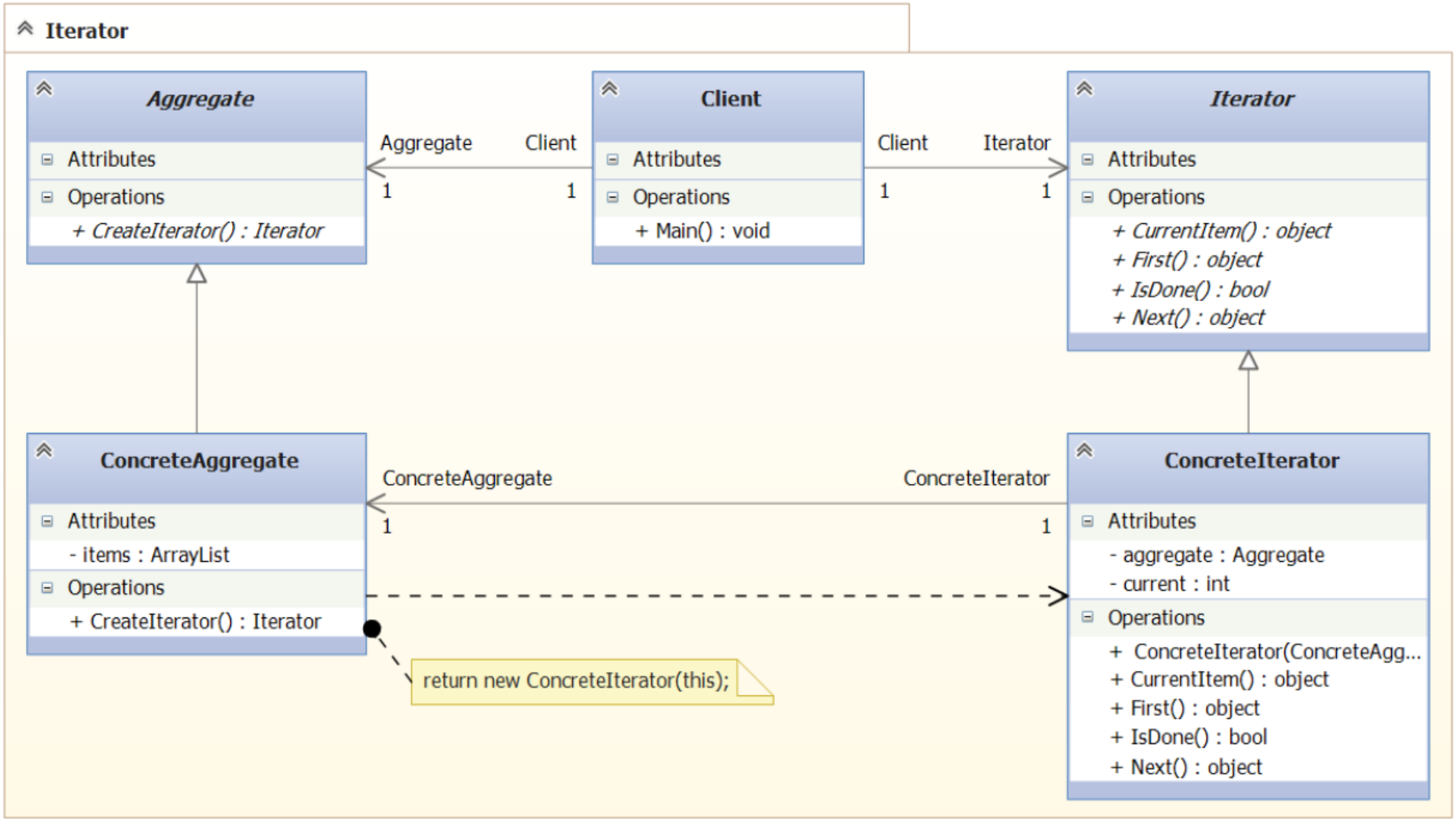
[2]**Конкретный итератор** реализует алгоритм обхода какой-то конкретной коллекции. Объект итератора должен сам отслеживать текущую позицию при обходе коллекции, чтобы отдельные итераторы могли обходить одну и ту же коллекцию независимо..

[3]**Коллекция** описывает интерфейс получения итератора из коллекции. (коллекция: список, дерево, граф, хеш-таблица…) .
Коллекция может создавать итераторы, устанавливая, какие именно итераторы способны с ней работать.

[4]**Конкретная коллекция** возвращает новый экземпляр определённого конкретного итератора, связав его с текущим объектом коллекции. Сигнатура метода возвращает интерфейс итератора. Это позволяет клиенту не зависеть от конкретных классов итераторов.

[5]**Клиент** работает со всеми объектами через интерфейсы коллекции и итератора. Так клиентский код не зависит от конкретных классов, что позволяет применять различные итераторы, не изменяя существующий код программы.
В общем случае клиенты не создают объекты итераторов, а получают их из коллекций. Тем не менее, если клиенту требуется специальный итератор, он всегда может создать его самостоятельно.

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Aggregate a = new ConcreteAggregate();

        a[0] = "Element A";
        a[1] = "Element B";
        a[2] = "Element C";
        a[3] = "Element D";

        Iterator i = a.CreateIterator();

        for (object e = i.First(); !i.IsDone(); e = i.Next())
            Console.WriteLine(e);
    }
}
```

`Aggregate` (`IEnumerable`) - **Агрегат**: Предоставляет интерфейс коллекции (набор методов) в том числе методы для создания объекта-итератора.

```
abstract class Aggregate
{
    public abstract Iterator CreateIterator();
    public abstract int Count { get; }
    public abstract object this[int index]
    {
       get;
       set;
    }
}
```

`Iterator` (`IEnumerator`) - **Итератор**: Предоставляет интерфейс (набор методов) для доступа к коллекции и обхода элементов.

```
abstract class Iterator
{
    public abstract object First();
    public abstract object Next();
    public abstract bool IsDone();
    public abstract object CurrentItem();
}
```

`ConcreteAggregate` - **Конкретный агрегат**: Реализует интерфейс коллекции и хранит в себе элементы.

```
class ConcreteAggregate : Aggregate
{
    private ArrayList items = new ArrayList();

    public override Iterator CreateIterator()
    {
        return new ConcreteIterator(this);
    }

    public override int Count
    {
        get { return items.Count; }
    }

    public override object this[int index]
    {
        get { return items[index]; }
        set { items.Insert(index, value); }
    }
}
```

```
class ConcreteIterator : Iterator
{
    private Aggregate aggregate;
    private int current = 0;

    public ConcreteIterator(Aggregate aggregate)
    {
        this.aggregate = aggregate;
    }

    public override object First()
    {
        return aggregate[0];
    }

    public override object Next()
    {
        if (current++ < aggregate.Count - 1)
            return aggregate[current];
        else
            return null;
    }

    public override object CurrentItem()
    {
        return aggregate[current];
    }

    public override bool IsDone()
    {
        if (current < aggregate.Count)
            return false;

        current = 0;
        return true;
    }
}
```

`ConcreteIterator` - **Конкретный итератор**: Реализует интерфейс класса `Iterator`. Следит за позицией текущего элемента при переборе коллекции (`Aggregate`).

Отношения между участниками

Отношения между классами

- Конкретный класс ConcreteAggregate связан связью отношения наследования с абстрактным классом Aggregate и связью отношения зависимости с конкретным классом ConcreteIterator.
- Конкретный класс ConcreteIterator связан связью отношения наследования с абстрактным классом Iterator и связью отношения ассоциации с конкретным классом ConcreteAggregate.

Отношения между объектами

- ConcreteIterator отслеживает текущий элемент в коллекции (Aggregate) и может вычислить следующий за ним элемент.

Применение

1. Когда имеется сложная структура данных, и необходимо скрыть от клиента детали её реализации (из-за сложности или вопросов безопасности).

Итератор предоставляет клиенту всего несколько простых методов перебора элементов коллекции. Это не только упрощает доступ к коллекции, но и защищает её данные от неосторожных или злоумышленных действий.

2. Когда необходимо иметь несколько вариантов обхода одной и той же структуры данных.

Нетривиальные алгоритмы обхода структуры данных могут иметь довольно объёмный код. Этот утяжеляет класс коллекции или часть бизнес-логики программы. Применив итератор, можно выделить код обхода структуры данных в собственный класс, упростив поддержку остального кода.

3. Когда необходимо реализовать единый интерфейс обхода различных структур данных.

Итератор позволяет вынести реализации различных вариантов обхода в подклассы. Это позволит легко взаимозаменять объекты итераторов, в зависимости от того, с какой структурой данных приходится работать.

Шаги реализации

1.Создать общий интерфейс итераторов. Обязательный минимум — это операция получения следующего элемента коллекции. Но для удобства можно предусмотреть методы для получения предыдущего элемента, текущей позиции, проверки окончания обхода и прочие.

2.Создать интерфейс коллекции и описать в нём метод получения итератора. Важно, чтобы сигнатура метода возвращала общий интерфейс итераторов, а не один из конкретных итераторов.

3.Создать классы конкретных итераторов для тех коллекций, которые нужно обходить с помощью паттерна. Итератор должен быть привязан только к одному объекту коллекции. Обычно эта связь устанавливается через конструктор.

4.Реализовать методы получения итератора в конкретных классах коллекций. Они должны создавать новый итератор того класса, который способен работать с данным типом коллекции. Коллекция должна передавать ссылку на собственный объект в конструктор итератора.

5.В клиентском коде и в классах коллекций не должно остаться кода обхода элементов. Клиент должен получать новый итератор из объекта коллекции каждый раз, когда ему нужно перебрать её элементы.

Преимущества и недостатки

+ Упрощает классы хранения данных.
+ Позволяет реализовать различные способы обхода структуры данных.
+ Позволяет одновременно перемещаться по структуре данных в разные стороны.
- Не оправдан, если можно обойтись простым циклом.